Week 7

Agenda

- Exam grades
- HW3 due next week (November 3rd)
- 2 more lectures – exam 2

Decision Trees and Overfitting

Decision trees are built using training data and the structure of the tree can change with new or a different dataset.

A fully grown tree tends to perfectly fit the training data

This can lead to excellent results on the training data but not as good results when used on new data

Decision trees tend to ***overfit*** the training data

Ways to Reduce Overfitting of Decision Trees

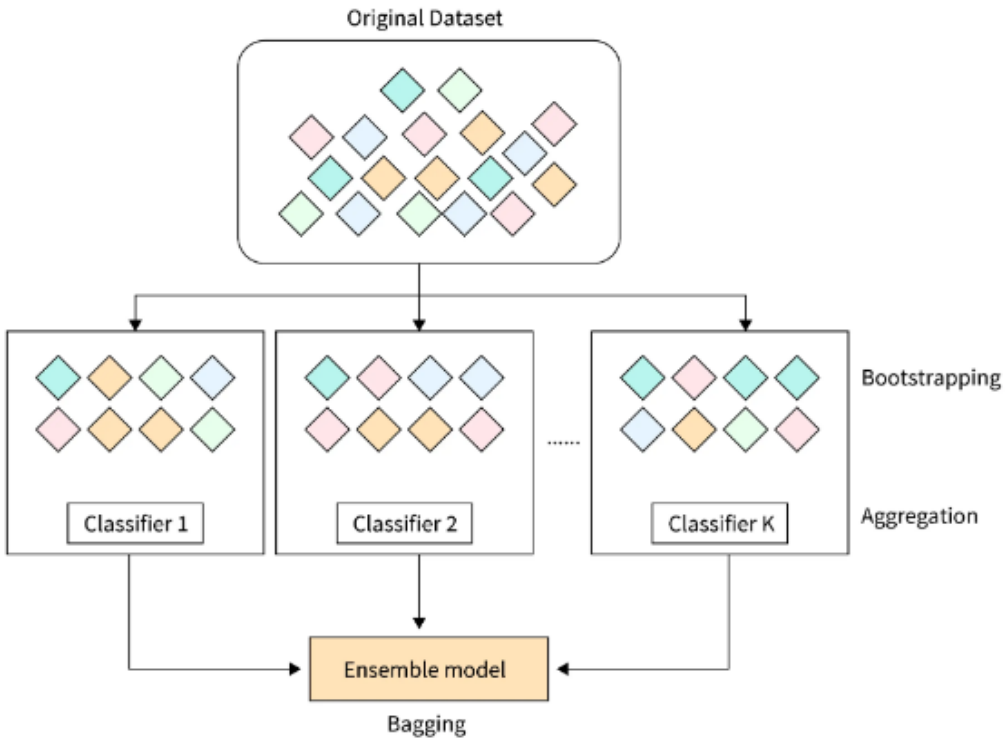
1. Pruning/Regularization – see last week's lecture
2. Bagging/Boosting – used very successfully with decision trees

Bagging

- Bagging is an **ensemble learning method** that combines predictions from multiple models trained on different **bootstrap samples** of the training data (samples drawn *with replacement*).

- In this approach we generate B different bootstrapped training data sets (defined in next slide) called bags. We then train B different models on each bag. We then average all the predictions (regression problems) or take a majority vote (classification problems) from the B models to obtain a single prediction.

- We will have B bags and B different decision trees.



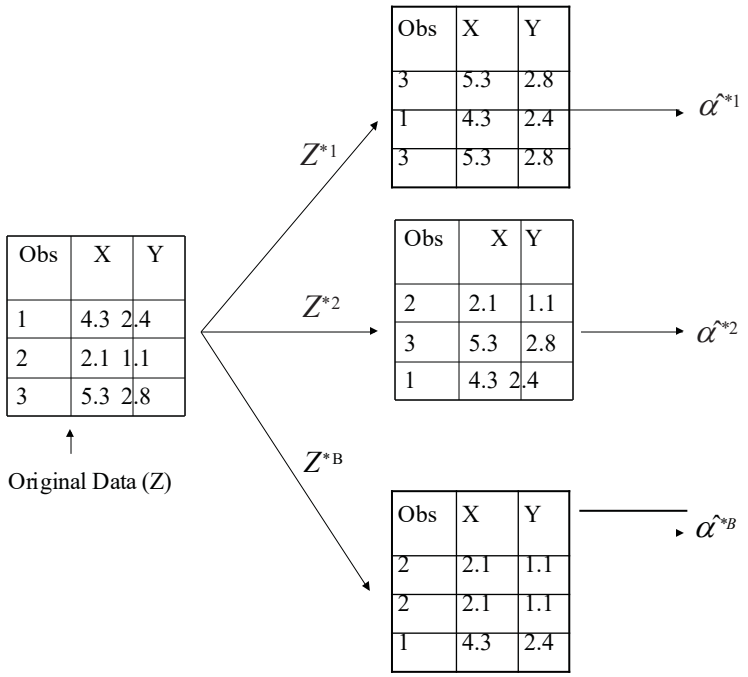
Bagging – how to come up with different training data sets

- The bootstrap approach allows us to train our model on multiple different data sets. How?

- Rather that obtaining several new data sets from the population, we instead obtain distinct data sets by repeatedly sampling observations from the original dataset with replacement.

- Each of these "bootstrap data sets" is created by sampling with replacement, and is the same size as our original dataset. As a result some observations may appear more than once in a given bootstrap data set and some not at all.

Example – sample with replacement



Notice observation 3 shows up twice in the first sample. This is sampling with replacement.

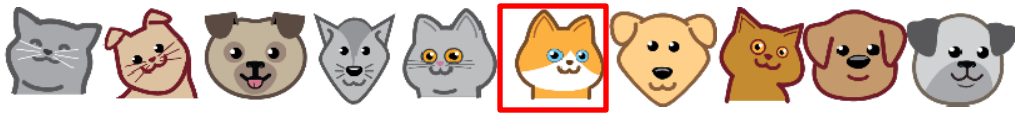
The example shows 3 bootstrap datasets.

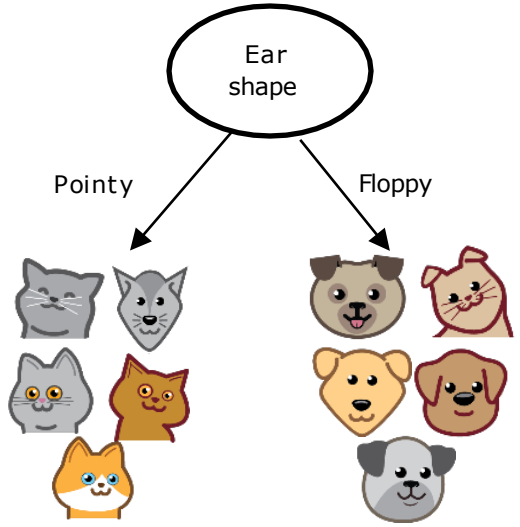
Bootstrap data sets are the same size as the original data set.

Sampling with replacement randomly selects samples from the original data set.

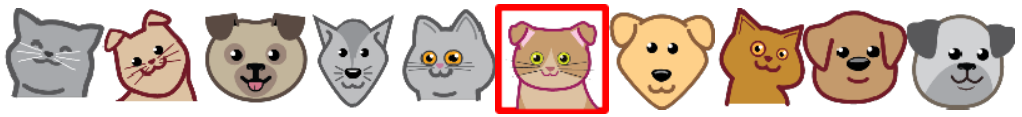
Replacement means that once a sample is randomly selected it can be selected again. It is not removed from the selection process.

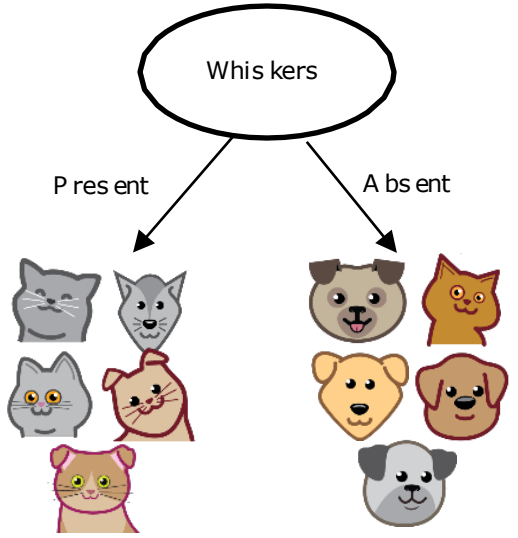
Trees are highly sensitive to small changes of the data. In the example below, one sample in the training set changes and a whole new tree is built.





Change one sample and a brand new tree with a different root is built.



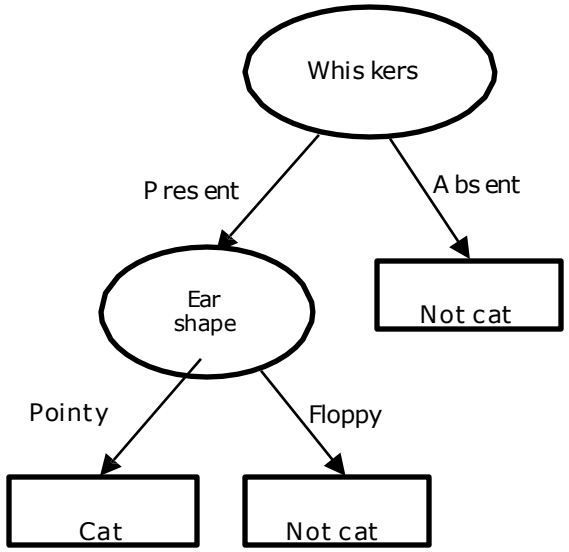


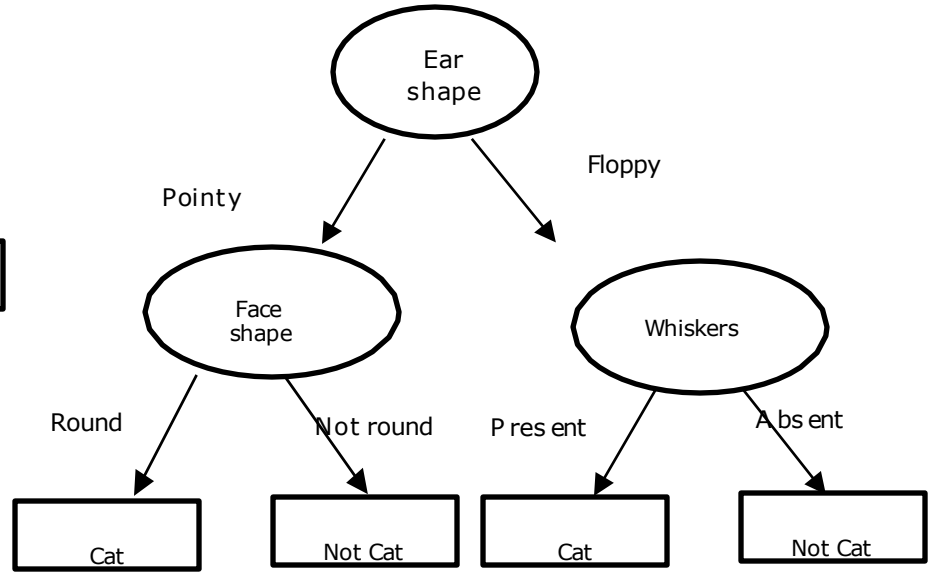
Bagging tree predictions

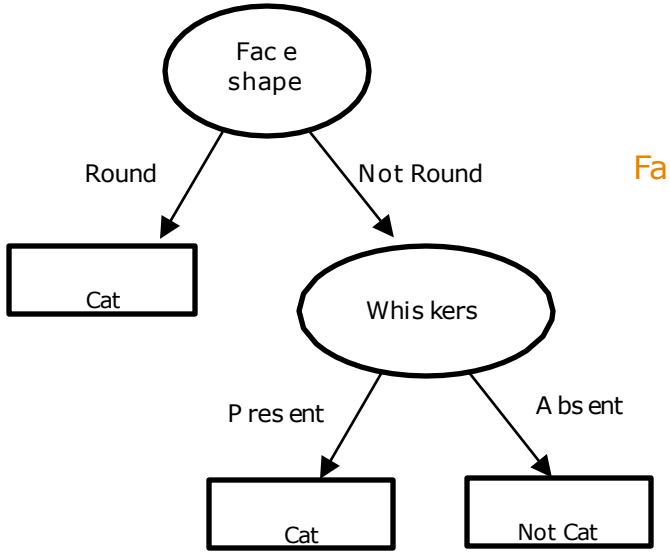
- Regression trees take the average of each prediction made by the B models.

- For classification trees: for each test observation, we record the class predicted by each of the *B* trees, and take a *majority vote*: the overall prediction is the most commonly occurring class among the *B* predictions.

The b number of trees created using bagging is called a tree ensemble



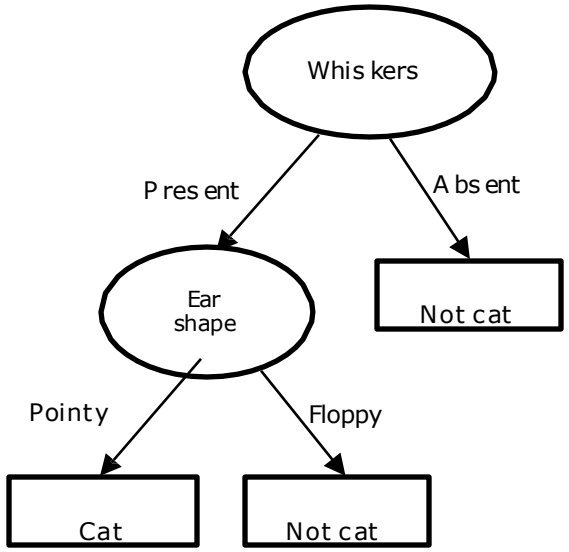




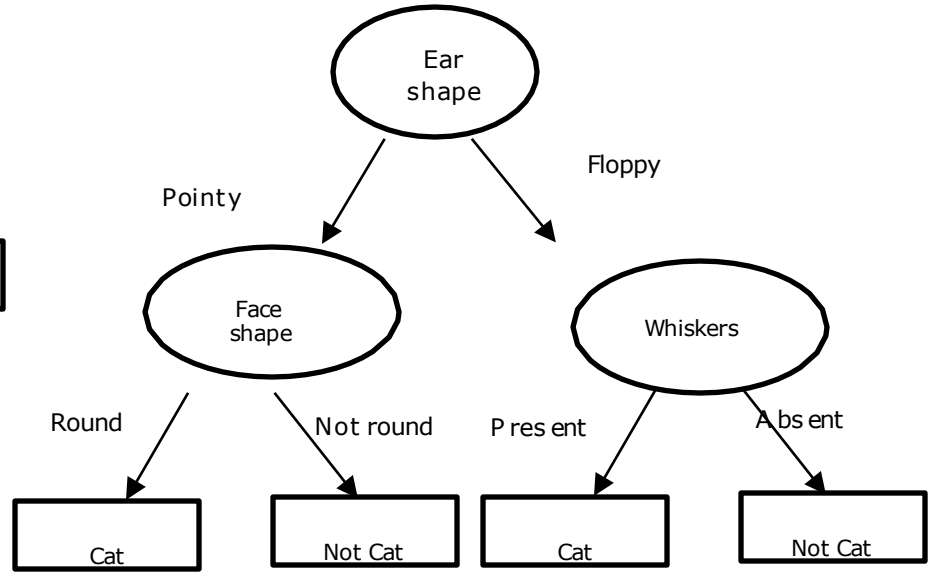
New test example

Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

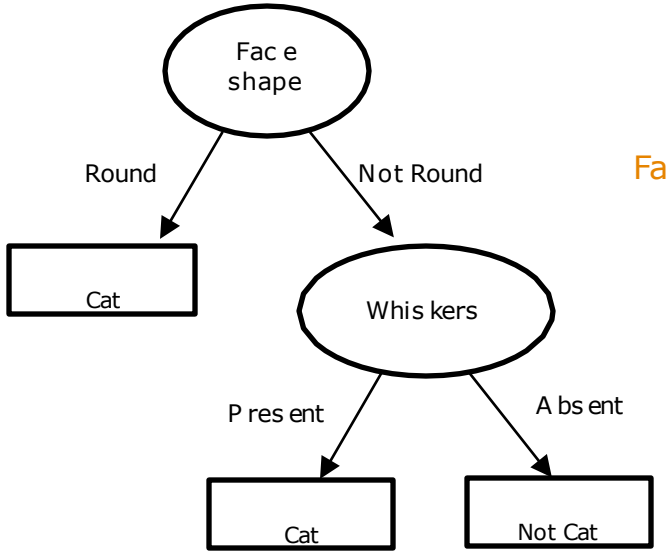
The b number of trees created using bagging is called a tree ensemble



Prediction: Cat



Prediction: Not cat



Prediction: Cat

New test example

Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

Final prediction: Cat

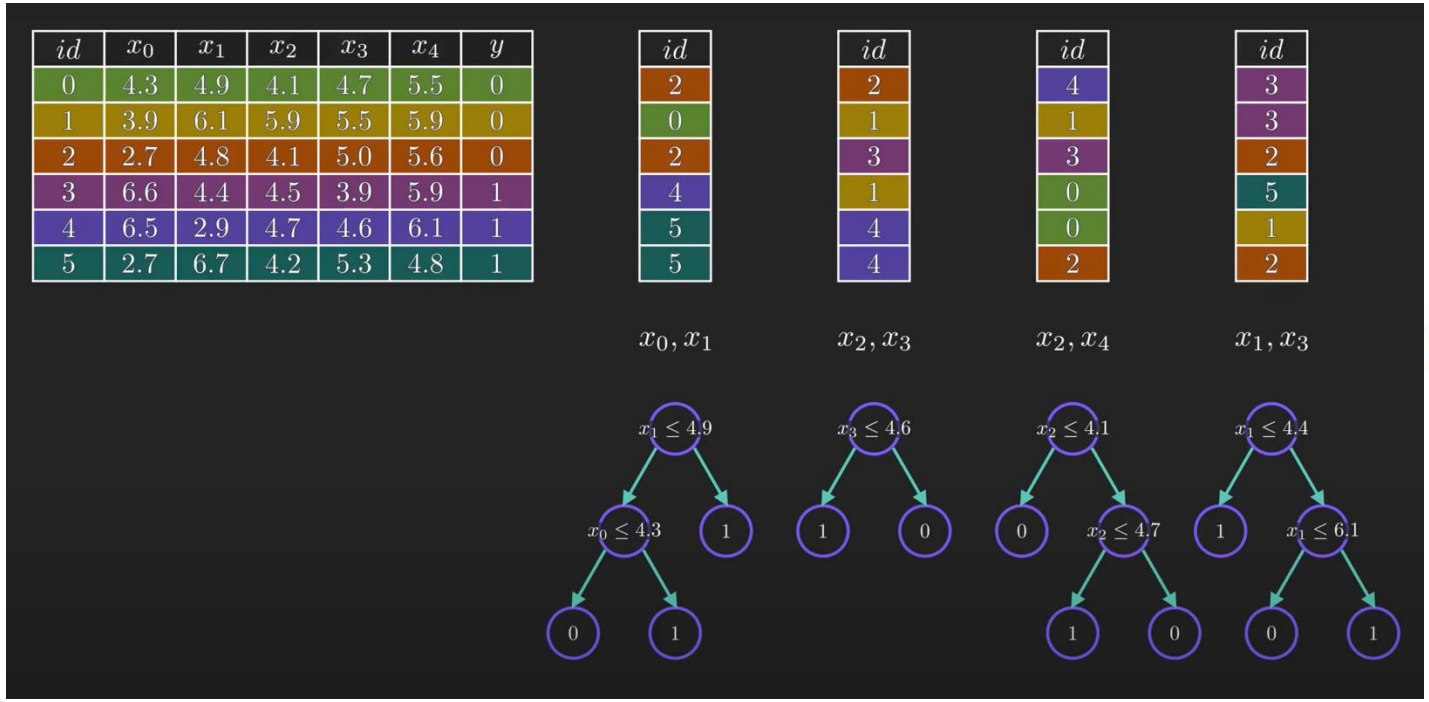
Random Forrest

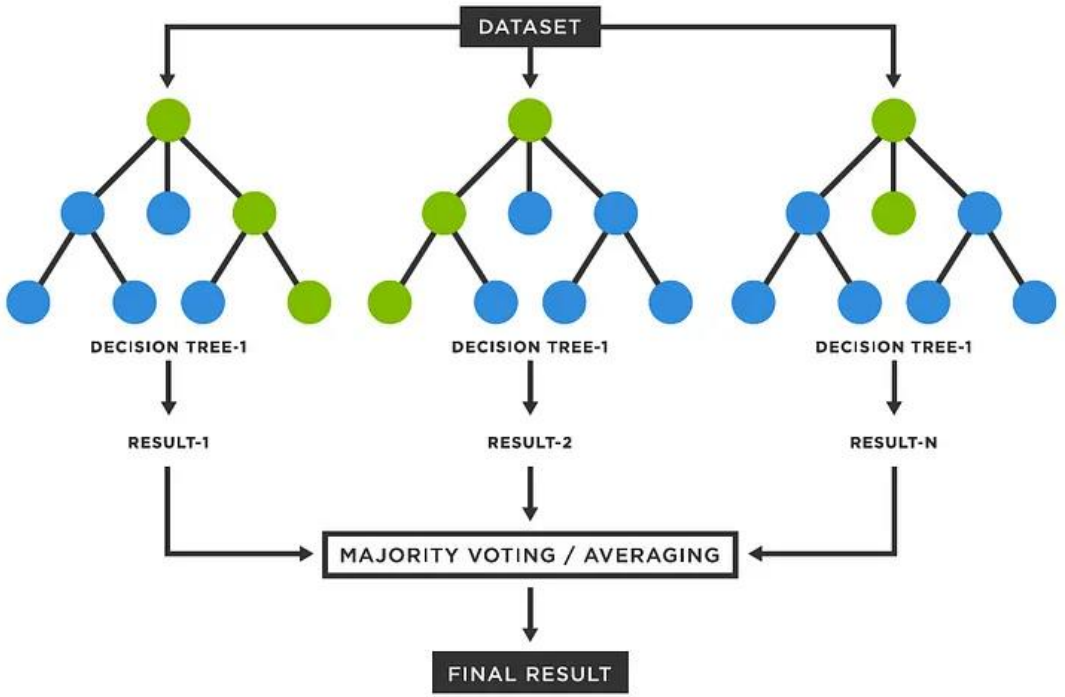
Example of Bagging: Random Forests

- *Even with bagging, the various trees built can still be quite similar.*
- *Random forests can* provide an improvement over bagged trees by way of a small tweak that *decorrelates* the trees. Meaning that each tree will be unique.
- As in bagging, we build a number of decision trees on bootstrapped training samples.
- But when building these decision trees, each time a split in a tree is considered, *a random selection of m features* is chosen as split candidates from the full set of *features*.
- A fresh selection of *k features* is taken at each split, and typically *k* is approximately equal to the square root of the total number of features.

Random Forest

- The "randomness" comes from two processes: each tree is built from a different random bootstrap sample of the data, and at each node, only a random subset of features is considered for splitting





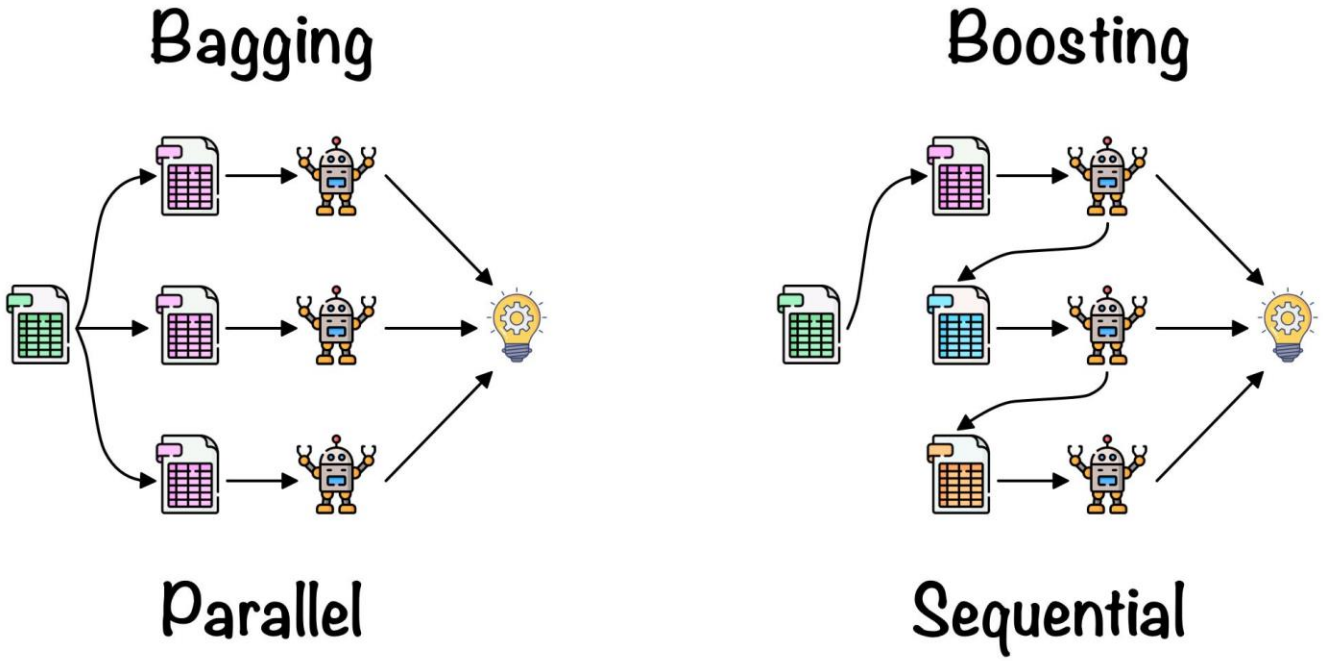
Randomizing the feature choice

At each node, when choosing a feature to use to split, if n features are available, pick a random subset of $k < n$ features and allow the algorithm to only choose from that subset of features.

$$k = \sqrt{n}$$

Boosting

- Boosting is an ensemble learning technique in machine learning that combines a set of "weak learners" (models that perform slightly better than random guessing) to create a single, highly accurate "strong learner."



Boosting – core idea

1. Train a tiny model.
2. See where it errors.
3. Train the next tiny model to fix those errors.
4. Repeat and add them up.

- There are different algorithms under the boosting family, such as:
  - AdaBoost – >uses error samples to guide the next tree
  - XGBoost, LightGBM, CatBoost –> uses gradients of a loss function to guide the next tree

XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)

XGBoost (eXtreme Gradient Boosting)

Classification

```
from xgboost import XGBClassifier

model = XGBClassifier()


model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Regression

```
from xgboost import XGBRegressor

model = XGBRegressor()


model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

When to use decision trees?

Decision Trees vs Neural Networks

Decision Trees and Tree ensembles

- Works well on tabular (structured) data
- Not recommended for unstructured data (images, audio, text)
- Fast
- Small decision trees may be human interpretable. Note that tree ensembles are not and are considered black box algorithms.

Neural Networks

- Works well on all types of data, including tabular (structured) and unstructured data
- May be slower than a decision tree
- Works with *transfer learning* – e.g. can use an already trained image classifier and use those parameters for the first epoch of training for a different image classification problem. Speeds up training for new problem.